REPORT · 2026年10月17日

稠密阶段耗时归因与并发互删诊断

H3D v1.1.0 · r6.2 实机复盘

文保数字化研发组
2026年10月17日

目 录

一、结论摘要 4

1.1 六条根因一览 4

二、本次运行时间线（一手数据） 5

2.1 单张深度图耗时的「爬升」现象 5

三、R1：两个批处理进程并发互删（首要原因） 6

3.1 证据链 6

3.2 为什么「同时跑」比「跑两遍」更糟 6

四、R2：4 尊造像 = 同一批照片 7

五、R3~R6：稠密阶段自身的四层浪费 8

5.1 两遍稠密（geom_consistency） 8

5.2 分辨率：2400 长边的代价 8

5.3 超时预算写死 3600 s 8

5.4 超时后 43 分钟成果被删 8

六、为什么失败原因写着「需要 CUDA GPU」 9

七、已修复项（r6.2） 10

7.1 F1 单实例锁 10

7.2 F2 超时预算 10

7.3 F3 位姿指纹与断点续算 10

7.4 F6 超时文案（真实输出） 11

八、校验结果 12

8.1 单元测试 12

8.2 端到端冒烟（真实 COLMAP） 12

8.3 配置兼容性 12

九、复跑建议 13

9.1 先做三件小事 13

9.2 参数选择 13
9.3 断点续算怎么用 13
十、遗留风险与下一步 14

一、结论摘要

2026-10-17 21:40—23:00 的实机运行，用户等待 **1 小时 30 分**、取消后仍未见任何成果。经逐层核对日志、目录产物、进程列表与 run_manifest，结论是：**管线本身没有算错，而是四重浪费叠加**——同一批 30 张照片被 4 尊造像重复计算、两个批处理进程并发互删中间产物并争抢 GPU、稠密阶段跑两遍却只给了一遍的超时预算、超时后已算完的 43 分钟深度图又被下一轮无条件清掉。

> **一句话归因**：用户等的 90 分钟里，真正「有产出」的计算只有 43 分 35 秒（30 张第一遍深度图）；其余时间消耗在重复重建（×4 份）、双进程抢 GPU（单张 45 s → 87~145 s）、第二遍几何校验（再 ×1.6），最后在 3600 s 超时处被腰斩，并且这份 43 分钟的成果**没有保住**。

1.1 六条根因一览

序号	根因	量化影响	级别
R1	两个批处理进程并发同跑（PID 26048 / 26332）	互删 dense 目录致 69 min 作废；GPU 争抢使单张 45 s → 87/145 s（2~3 倍）	致命
R2	4 尊造像使用同一批 30 张照片（DSC_0008~0037）	计算量 ×4；三份 sparse 模型逐位相同（Points 13990）	致命
R3	patch_match_stereo 跑两遍（geom_consistency=true）	第二遍 30 张另需约 70 min，总需求 6815 s	高
R4	稠密分辨率沿用 2400 长边（3.84 MP）	深度图像素数 ×2.25 vs 1600 长边（1.71 MP）	高
R5	超时预算 3600 s 写死且覆盖整条命令	需求 6815 s > 预算 3600 s，**必然**腰斩	致命
R6	超时/崩溃后成果不可续算（无条件清 stereo）	30 张第一遍深度图（43 min）在下次运行时被删除	高

二、本次运行时间线（一手数据）

以下数据全部取自 batch_gui.log 时间戳，未做估算。以首尊 relic_001BMYyyyy 为例，从进入 S1 到被超时杀灭共 **4055 s（67.6 min）**：

阶段	起	止	耗时	说明
照片预处理 + 特征标定	21:40:03	21:40:11	8 s	30 张缩放到 2400 长边
COLMAP 特征提取	21:40:11	21:41:51	100 s	单线程确定性模式
穷举匹配 435 对	21:41:51	21:46:44	293 s	30 张 → exhaustive
mapper 稀疏重建	21:46:44	21:47:34	50 s	30/30 注册，1 相机
去畸变	21:47:34	21:47:37	3 s	image_undistorter
深度图第一遍（photometric）	21:47:37	22:31:12	2615 s	30 张，均 87 s/张
几何校验第二遍（geometric）	22:33:08	22:47:38	870 s	仅 6/30 张即被杀
合计	**21:40:03**	**22:47:38**	**4055 s**	**67.6 min**

2.1 单张深度图耗时的「爬升」现象

第一遍内部并非匀速：前 8 张 43~53 s，第 9 张起跳到 90~130 s，第二遍更是 145 s/张。这不是 COLMAP 越跑越慢，而是**另一个进程在同一块 GPU 上开始了 patch_match**：旧进程的 relic_666 在其日志里留下了 22:47:56 的写入时间戳，与本轮 22:47:38 的失败时刻重合，两者确实同时在跑稠密。

区段	单张耗时	相对基线	判读
第 1~8 张	43~53 s	≈1.0×（独占基线）	单进程、GPU 空闲
第 9~30 张	90~130 s	≈2.2×	第二个进程进入稠密，争抢 GPU
第二遍（前 6 张）	145 s/张	≈3.2×	双进程 + geom_consistency 本身更贵

三、R1：两个批处理进程并发互删（首要原因）

3.1 证据链

① **进程表**：核查时同机存在 3 个 H3D.exe（26628 / 26048 / 26332）与 2 个colmap.exe，其中 26048 是 20:36 那一轮、26332 是 21:39 这一轮——两轮都在跑。② **COLMAP 崩溃信息**：21:45:49 的 colmap_step12.log 末尾为<code>Could not create logging file: No such file or directory</code> 与<code>Check failed: (file).is_open() Could not open ...DSC_0037.JPG.geometric.bin</code>，这是 **cwd 与目标目录已被删除**的典型症状；而 21:40:11 新进程恰好打印了「S1 清理旧 dense 工作区」。③ **manifest 互写**：22:48:04 落盘的记录里，relic_001BMYyyyy 的 elapsed_s=4158、原因写着「需要 CUDA GPU」，与该轮实际的TimeoutExpired 完全不符——说明该条记录来自另一个进程，两个进程在无锁地覆写同一个 run_manifest.json。

观察点	数据	推断
同机进程	3× H3D.exe + 2× colmap.exe	至少两轮批处理同时在跑
旧轮 S1 起点	21:45:49 − 4158 s = 20:36:31	与用户 20:36 那次启动吻合
新轮清 dense	21:40:11	旧轮 patch_match 仍在写 → 目录消失
旧轮崩溃	21:45:49	写第 30 张 geometric.bin 失败
manifest mtime	22:48:04 但内容为旧轮记录	无锁覆写，记录互相污染

3.2 为什么「同时跑」比「跑两遍」更糟

串行跑 4 尊是 4 倍时间；并发跑同一批则是**互相破坏**：两边的<code>shutil.rmtree(dense)</code> 会删掉对方还没写完的深度图，<code>patch_match_stereo</code> 又都会「跳过已存在的 photometric.bin」，于是残留的不完整深度图会被当成有效结果参与融合——这正是 r4.1 里「静默错误点云」的同一机制，只是这次表现为崩溃而非脏数据。

四、R2：4 尊造像 = 同一批照片

00_photo 下 4 个目录（relic_001BMYyyyy / relic_666 / rrrrr / yyyyyyy）各有 30 张、文件名与尺寸完全一致（DSC_0008~DSC_0037，原始 4496×3000）。用 <code>colmap model_analyzer</code> 核对三份 sparse 模型，结果逐位相同：

造像	Cameras	Images	Points	Observations	平均重投影误差
relic_001BMYyyyy	1	30	13990	55242	0.496871 px
relic_666	1	30	13990	55242	0.496871 px
rrrrr	1	30	13990	55242	0.496871 px

三份模型完全相同 = 同一批照片被当成三尊文物各算了一遍（第四尊 yyyyyyy 因用户取消未跑到）。这既是 4 倍耗时，也说明「任务发现」缺少重复照片检测：系统应当提示「这 4 尊照片指纹相同，将重复重建 4 次」，而不是闷头全跑。

五、R3~R6：稠密阶段自身的四层浪费

5.1 两遍稠密（geom_consistency）

命令行为 <code>--PatchMatchStereo.geom_consistency true</code>。COLMAP 此时对每张图先算 photometric 深度图与法线图，全部完成后再做一遍跨视图几何一致性校验，写 <code>*.geometric.bin</code>。日志里 <code>Processing view n / N</code> 会出现2N 次——本轮 30 张即 60 次，第一遍 2615 s、第二遍约 70 min，合计 6815 s。第二遍的收益是边缘更干净、融合噪点更少；对「先跑通、先出成果」的阶段，完全可以只跑 photometric（fusion 改吃 photometric 深度图）。

5.2 分辨率：2400 长边的代价

配置	深度图分辨率	像素数	单遍估算	双遍估算
max_image_size=2400（本次）	2399×1601	3.84 MP	43.5 min	约 113 min
dense_max_image_size=1600（新默认）	1600×1068	1.71 MP	19 min	约 50 min
1600 + 单遍	1600×1068	1.71 MP	19 min	19 min

深度图像素数与 patch_match 耗时近似线性。稠密几何的作用是为纯照片造像提供点云底模（后续还要走 M3 泊松重建 + M4 照片贴图），1600 长边在 0.5 m 见方的造像上约 0.3 mm/px，与最终交付精度匹配；**稀疏解算仍需 2400**（特征点质量），两者本就该分开控制——原实现只有一个 max_image_size，稠密只能跟着吃 2400。

5.3 超时预算写死 3600 s

<code>run_dense_stereo</code> 里 <code>run_external(cmd, timeout=3600)</code> 覆盖的是**整条 patch_match_stereo 命令**，而该命令含两遍。实测需求 6815 s，预算 3600 s：**这是必然超时，不是偶发。**21:47:37 + 3600 s = 22:47:37，日志里 22:47:38 报 TimeoutExpired，一秒不差。

5.4 超时后 43 分钟成果被删

超时被杀时，depth_maps 下已有 35 个 bin（30 photometric + 5 geometric）。但下一轮 <code>run_dense_stereo</code> 入口无条件 <code>rmtree(stereo)</code>，这批成果会被整体删除。COLMAP 本身支持续算（跳过已存在的 photometric.bin），这恰恰是 r4.1 里需要防范的「静默错误」机制——问题不在 COLMAP，而在于我们既没利用它的续算能力，也没在位姿变化时正确作废旧结果。**正确做法是判据从「目录非空」改为「位姿指纹是否变化」。**

六、为什么失败原因写着「需要 CUDA GPU」

run_manifest.json 里两条失败记录的 reason 都是「dense 需要可用 CUDA GPU（patch_match_stereo 为 GPU-only）」，但真实原因是 TimeoutExpired。机制是：<code>run_external</code> 超时时**抛异常**而不是返回非零 returncode，于是 <code>_dump_step_log()</code> 根本没执行，<code>colmap_step12.log</code> 保留的是**上一轮**的旧内容；旧内容里满是 COLMAP 每次迭代都打印的<code>cudacc.cc:51</code>，而失败分支用 <code>if "CUDA" in err</code> 判定 GPU 问题，于是被旧日志误导。用户因此以为要装驱动，实际只需放宽预算或降分辨率。

> **教训**：异常路径同样要落盘证据；失败原因的判定不能基于「字符串里出现了 CUDA」，而应基于 stderr 中的真实错误行（<code>Check failed</code> / <code>cudaError</code>），并且必须限定在本轮日志内。

七、已修复项（r6.2）

编号	修复内容	涉及文件	对应根因
F1	数据根单实例锁（O_EXCL + PID 存活检测，崩溃残留自动接管）	core/lockfile.py（新）、runner/batch_processor.py	R1
F2	稠密超时改为「张数 × MP × 45 s × 遍数」动态预算，可显式覆盖	sfm_solve.dense_timeout_budget / core/config.py	R5
F3	位姿指纹判据：位姿未变复用深度图（断点续算），变了才清	sfm_solve._pose_fingerprint / run_dense_stereo	R6
F4	稠密分辨率独立配置 dense_max_image_size（默认 1600）	core/config.py / run_dense_stereo	R4
F5	dense_geom_consistency 可关（单遍），fusion 自动改吃 photometric	core/config.py / run_dense_stereo	R3
F6	超时异常携带已收集输出并落盘，失败文案给出「算到第几张 / 可否续算 / 怎么加速」	core/proc.py / run_dense_stereo	误诊
F7	取消时按进程树终止（taskkill /F /T），不再遗留 colmap 占 GPU	gui/runner.py	R1 残留

7.1 F1 单实例锁

```
from h3d.core.lockfile import acquire_run_lock, LockBusyError

try:
    lock = acquire_run_lock(data_root)     # <root>/.h3d_run.lock
except LockBusyError as exc:
    # 消息含占用者 PID 与「为什么不能同时跑」，直接展示给用户
    raise RecoverableError(str(exc)) from exc
try:
    ...                                    # 独占执行批处理
finally:
    lock.release()                         # 只删自己那份，不误删接管者
```

7.2 F2 超时预算

```
_PATCH_MATCH_S_PER_MP_PASS = 45.0   # 实测 22.7~37.8 s/MP，取 2 倍余量

def dense_timeout_budget(n_views, megapixels, passes=2):
    views = max(int(n_views), 1)
    mp = max(float(megapixels), 0.2)
    return int(max(1800, round(views * mp * 45.0 * max(passes, 1))))

# 30 张 3.84 MP 双遍 -> 10368 s（2.9 h，足够跑完）
# 30 张 1.71 MP 双遍 ->  4617 s
# 30 张 1.71 MP 单遍 ->  2308 s
```

7.3 F3 位姿指纹与断点续算

```
cur_fp = _pose_fingerprint(sparse_model)   # cameras/images/points3D 的 size+mtime
if has_stereo:
    if force_clear:            rmtree(stereo)   # force_redense 显式要求
    elif old_fp != cur_fp:     rmtree(stereo)   # 位姿变了 -> 旧深度图作废
    else:                      保留并记录「复用 N 张」   # 断点续算
写入 <dense>/.h3d_pose_fp
```

7.4 F6 超时文案（真实输出）

```
dense[2/3] 稠密深度图超时：预算 3600s 用尽（30 张 × 3.84 MP × 2 遍）。
已完成 photometric 30/30、geometric 6/30；产物已保留，重跑会从断点续算
（位姿未变时不再重算）。
加速建议：① 降低 sfm_solve.dense_max_image_size（当前 2400）；
        ② sfm_solve.dense_geom_consistency=false（省掉第二遍）；
        ③ 显式设置 sfm_solve.dense_timeout_s 放宽预算。
；完整日志：...\99_runs\sfm_tmp\<item>\logs\colmap_step12.log
```

八、校验结果

8.1 单元测试

测试集	项数	结果	覆盖的关键行为
test_run_lock.py（新增）	7	通过	二次获取被拒 / 陈旧锁接管 / 坏锁接管 / release 不误删
test_dense_budget.py（新增）	14	通过	预算公式 / 指纹稳定性 / 续算 / 强制清 / 超时文案 / 命令行参数
全量回归 tests/	568	通过	0 失败 0 错误（基线 547 + 新增 21）

8.2 端到端冒烟（真实 COLMAP）

用 3 张 1200 px 照片在独立数据根跑完整批次，验证新链路：

检查项	实测	结论
S1 总耗时	70.0 s（稀疏 + 双遍稠密）	新代码路径可跑通
深度图	3 photometric + 3 geometric	两遍均完成，geom_consistency 生效
位姿指纹	cameras.bin:64:...|images.bin:206615:...	指纹落盘正常
dense 点云	62268 点 → fused_dense.ply 1.43 MB	M1 正常消费
锁	运行结束无 .h3d_run.lock 残留	异常路径也能释放
M1→M4	预处理好 / 泊松 28349 面 / 照片贴图导出 obj+glb	下游全链路无回归
M5	coverage=6.3% < 80% → QualityGateError	3 张照片覆盖率不足，**预期失败**，非回归

8.3 配置兼容性

用户既有 config.yaml（无新键）载入后，新字段自动取默认值：dense_max_image_size=1600、dense_timeout_s=0（自动）、dense_geom_consistency=true、force_redense=false。**无需手工改配置**，稠密分辨率即从 2400 降到 1600。

九、复跑建议

9.1 先做三件小事

① **只留一尊**：把 00_photo 下重复的 relic_666 / rrrrr / yyyyyyy 移走，只保留 relic_001BMYyyyy，避免 4 倍重复计算；② **确认无残留进程**：任务管理器里不应再有 H3D.exe / colmap.exe（本次核查时曾有 3+2 个在跑）；③ **确认没有第二个窗口在跑同一目录**——新版本会直接拒绝启动并提示占用者 PID。

9.2 参数选择

目标	配置	单尊预计耗时（30 张，RTX 3050 独占）
先跑通（推荐）	dense_max_image_size=1600 + dense_geom_consistency=false	约 25~30 min
质量优先	dense_max_image_size=1600 + geom_consistency=true	约 50~55 min
极致质量	dense_max_image_size=2400 + geom_consistency=true	约 110~120 min

三档的 sparse 阶段（特征 100 s + 匹配 293 s + mapper 50 s）基本不变，约 7.5 min；差别全在稠密。第一档已足以支撑 M1-M5 出成果，后续需要精修时再单独调高 <code>force_redense=true</code> 重算稠密即可（稀疏结果会复用，不必重跑）。

9.3 断点续算怎么用

超时、断电、手动取消之后**不要删 99_runs**，直接重跑：位姿指纹未变，已完成的深度图会保留，COLMAP 只补缺的几张。日志里会出现「S1 复用既有深度图（位姿未变，断点续算）：photometric N 张 / geometric M 张」。只有当你改动了照片、重跑了 sparse、或显式设 <code>force_redense=true</code> 时，才会整棵清空重算。

十、遗留风险与下一步

编号	事项	影响	建议时机
P1	任务发现缺少「重复照片」检测	同一批照片被当多尊重复重建（本次 ×4）	下一轮
P1	dense 失败后无几何兜底	纯照片造像稠密失败 → 整尊 0 产出	下一轮
P2	稀疏阶段频繁 structure-less fallback 注册	30/30 注册成功但位姿约束弱，稠密可能有漂移	引入真实多角度照片时复核
P2	manifest 无锁非原子写	多进程/多 worker 时仍可能互相覆盖	与 F1 配套加文件锁
P3	M5 覆盖率阈值 80% 对纯照片链路偏严	3 张冒烟 coverage=6.3% 直接判失败	按采集规模分级

本轮交付边界：已修复 R1/R3/R4/R5/R6（并发锁、两遍可关、分辨率解耦、动态预算、断点续算、超时可诊断、取消杀进程树），并完成 568 项测试与真实 COLMAP 冒烟。「重复照片检测」与「稠密失败兜底」列为下一轮，不阻塞当前复跑。